

ЦОИ — ЛР2 — Вариант 5

Контурный анализ и сегментация: 1) выделение внутренней области порогом + связывание маской + очистка 2) выделение контура (Sobel) и "доработка" контура 3) дескрипторы Фурье для контура (инвариантность к сдвигу/масштабу/повороту)

```
clear; close all; clc;

thisDir = fileparts(mfilename("fullpath"));
imgDir = fullfile(thisDir, "Рисунки");
imgPath = fullfile(imgDir, "Img_05.jpg");
outDir = fullfile(thisDir, "Результаты");
if ~exist(outDir, "dir"); mkdir(outDir); end
```

Загрузка и подготовка

```
Irgb = imread(imgPath); % исходное изображение (RGB)
I = im2double(Irgb); % перевод в double [0..1]
if ndims(I) == 3 % если изображение цветное
    Ig = rgb2gray(I); % берём яркость (полутоновое)
else % если уже полутоновое
    Ig = I; % используем как есть
end

% Для пороговой обработки полезно повысить контраст (самолёт светлый)
IgAdj = adapthisteq(Ig); % локальное выравнивание гистограммы (мягко)
```

1) Выделение внутренней области порогом

Автопорог по Оцу + небольшая подстройка (часто требуется на реальных фото)

```
T0 = graythresh(Ig); % Otsu по исходной яркости (более стабильно для крыльев)
T = min(max(T0 - 0.05, 0), 1); % подстройка: понижаем порог, чтобы крылья попали в объект

BWgray = Ig >= T; % бинаризация по яркости: светлое -> объект

% Замыкание до реконструкции: помогает "сшить" фюзеляж и оба крыла при тенях
BWgray = imclose(BWgray, strel("disk", 5));

% Отделение самолёта от фона делаем через морфологическую реконструкцию:
% 1) эрозией разрываем тонкие "мостики" к фону
% 2) берём компоненту, содержащую центр изображения
% 3) восстанавливаем её до исходной BWgray через imreconstruct
seBreak = strel("disk", 2);
BWer = imerode(BWgray, seBreak);
[m0, n0] = size(BWer);
cx = round(n0/2); cy = round(m0/2);
BWseed0 = bwselect(BWer, cx, cy, 8);
if ~any(BWseed0(:))
    BWseed0 = keepBestComponent(BWer);
```

```

end
BW0 = imreconstruct(BWseed0, BWgray);

% Небольшое замыкание, чтобы соединить части самолёта после реконструкции
BW0 = imclose(BW0, strel("disk", 4));

% Связывание области "маской" как в методичке: если в окне есть >=k единиц -> ставим 1
g = 3; % половина размера окна (окно = (2g+1)x(2g+1))
kMin = 6; % порог суммы внутри окна (для 7x7: 0..49)

BWlink = linkByMask(BW0, g, kMin);

% Очистка: убрать мелкий шум и заполнить дырки
BWclean = imopen(BWlink, strel("disk", 2)); % убрать тонкие фоновые линии/полосы
BWclean = bwareaopen(BWclean, 300); % удалить мелкие компоненты (порог по площади)
BWclean = imfill(BWclean, "holes"); % заполнить дырки внутри объекта
BWclean = imclose(BWclean, strel("disk", 6)); % замкнуть разрывы по границе

% Частая проблема на фото: к объекту "прилипает" длинная тонкая область (дорога/линия).
% Удаляем такие придатки: открытие даёт "маркер" без тонких выступов, реконструкция возвращает
% но не восстанавливает тонкие мостики/дороги.
marker = imopen(BWclean, strel("disk", 8));
BWclean = imreconstruct(marker, BWclean);

% На всякий случай оставляем компоненту, ближайшую к центру (самолёт).
BWclean = keepBestComponent(BWclean);

% Если "дорога" всё ещё прилипла, отрезаем её так:
% - слегка эродируем, чтобы разорвать тонкие соединения
% - выбираем компоненту по центральному seed
% - реконструируем обратно (дорога не восстановится, если она отвалилась на шаге эрозии)
seCut = strel("disk", 3);
BWcut = imerode(BWclean, seCut);
[mC, nC] = size(BWclean);
BWseedC = bwselect(BWcut, round(nC/2), round(mC/2), 8);
if any(BWseedC(:))
    BWclean = imreconstruct(BWseedC, BWclean);
end

% Последний шаг против "дороги": убираем тонкие дальние придатки.
% Идея: у дороги малая "толщина" (distance transform), у самолёта – больше.
CCc = bwconncomp(BWclean);
if CCc.NumObjects > 0
    st = regionprops(BWclean, "Centroid");
    c = st.Centroid;
    [Xg, Yg] = meshgrid(1:nC, 1:mC);
    R = hypot(Xg - c(1), Yg - c(2)); % расстояние до центра
    D = bwdist(~BWclean); % "толщина" объекта
    Rth = 0.40 * hypot(nC, mC); % дальняя зона (чуть ближе, чтобы резать "хвосты")
    BWclean((D <= 4) & (R > Rth)) = 0; % убрать тонкие/средние пиксели далеко от центра

```

```

BWclean = imclose(BWclean, strel("disk", 5)); % восстановить край
BWclean = imfill(BWclean, "holes"); % заполнить дырки
end

% Если на крыле всё ещё остаётся диагональная "дорога", убираем её как линейную структуру.
% Делаем это осторожно: вычитаем только тонкие пиксели, чтобы не срезать крыло.
road = imopen(BWclean, strel("line", 35, -45)); % диагональ вверх-вправо (дорога)
Dthin = bwdist(~BWclean); % толщина объекта (в пикселях)
BWclean = BWclean & ~(imdilate(road, strel("disk", 9)) & (Dthin <= 5)); % убрать тонкую диагональ
BWclean = imclose(BWclean, strel("disk", 3)); % восстановить край после вычитания
BWclean = imfill(BWclean, "holes"); % заполнить дырки

```

2) Контур (Sobel) и доработка

```

edgeT = 0.12; % порог чувствительности Sobel (подбирается)
BWedge = edge(BWclean, "sobel", edgeT); % контур области

% Периметр области обычно даёт более "ровную" границу, чем Sobel на бинарной маске.
% Для стабильности берём объединение Sobel и периметра.
BWper = bwperim(BWclean, 8);
BWedge = BWedge | BWper;

% Доработка контура: замыкание разрывов и тонкая очистка
BWedge2 = bwmorph(BWedge, "bridge");
BWedge2 = bwmorph(BWedge2, "close");
BWedge2 = bwmorph(BWedge2, "thin", Inf);
BWedge2 = bwmorph(BWedge2, "spur", 10); % убрать "усики" на контуре
BWedge2 = bwmorph(BWedge2, "clean"); % убрать одиночные пиксели
BWedge2 = bwmorph(BWedge2, "thin", Inf); % снова сделать контур однопиксельным
BWedge2 = bwareaopen(BWedge2, 50);

% Страховка: если контур пропал, берём периметр области.
if bwconncomp(BWedge2).NumObjects == 0
    BWedge2 = bwperim(BWclean, 8);
end

```

3) Дескрипторы Фурье для контура

Берём самый большой замкнутый контур, представляем как комплексный сигнал $x+iy$

```

[B, z] = contourToComplex(BWedge2);
FD = abs(fft(z)); % амплитудный спектр — дескрипторы
FD = FD / max(FD(:) + eps); % нормировка (инвариантность к масштабу)

% Для отчёта обычно показывают первые N коэффициентов
Nshow = 80;

```

Проверка инвариантности к повороту

```

angles = 0:45:315;

```

```

FD_rot_all = cell(length(angles),1);

for i = 1:length(angles)
    ang = angles(i);

    % Поворот бинарного изображения (контурного объекта)
    BWrot = imrotate(BWclean, ang, "bilinear", "crop");

    % Берём контур заново
    BWedge_rot = edge(BWrot, "sobel", edgeT);
    BWedge_rot = BWedge_rot | bwperim(BWrot, 8);

    BWedge_rot = bwmorph(BWedge_rot, "bridge");
    BWedge_rot = bwmorph(BWedge_rot, "thin", Inf);

    % Если контур пустой – пропуск
    if bwconncomp(BWedge_rot).NumObjects == 0
        continue;
    end

    % Получаем контур как комплексный сигнал
    [~, zrot] = contourToComplex(BWedge_rot);

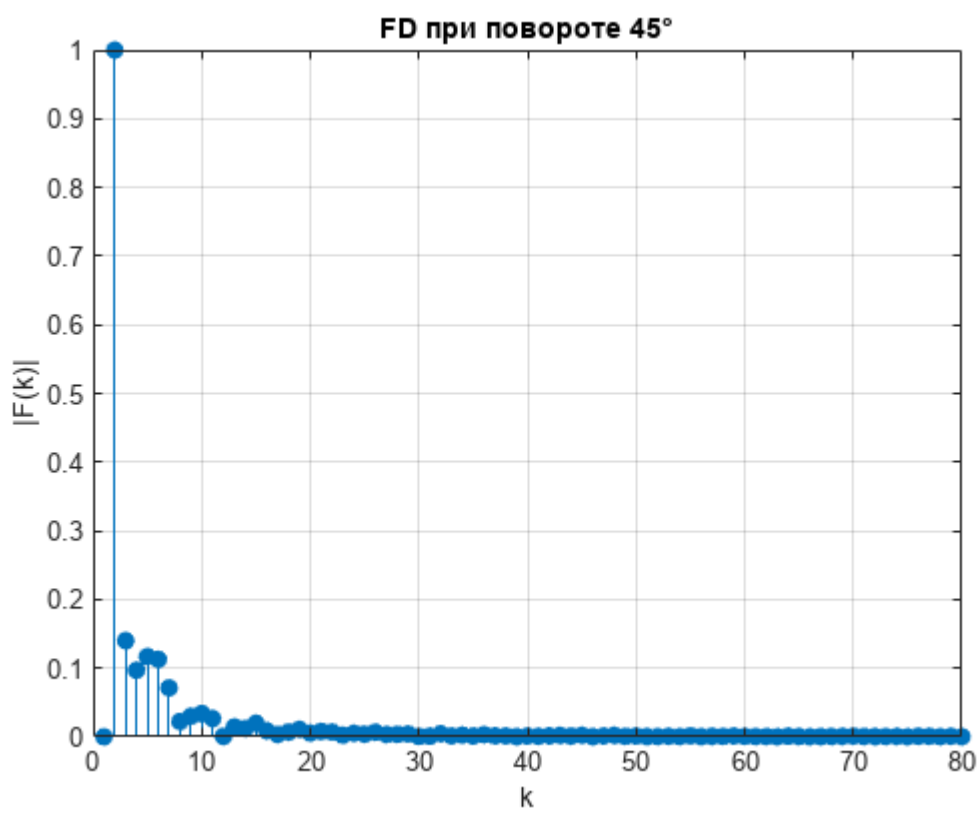
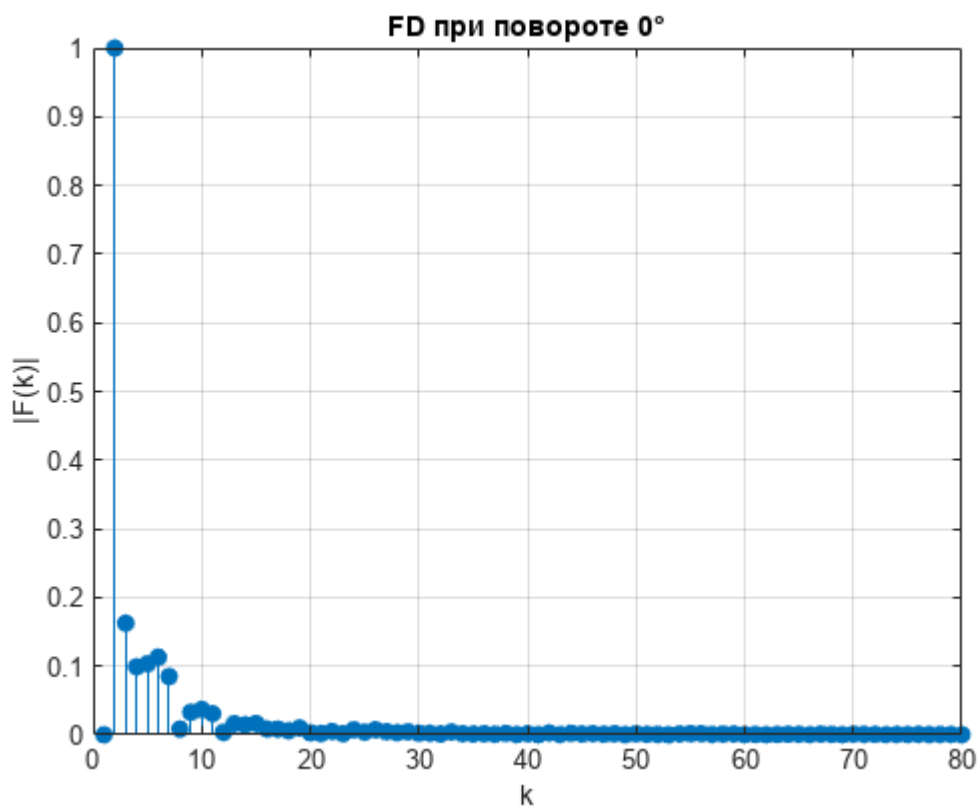
    % Фурье-дескрипторы
    FD_rot = abs(fft(zrot));
    FD_rot = FD_rot / max(FD_rot(:) + eps);

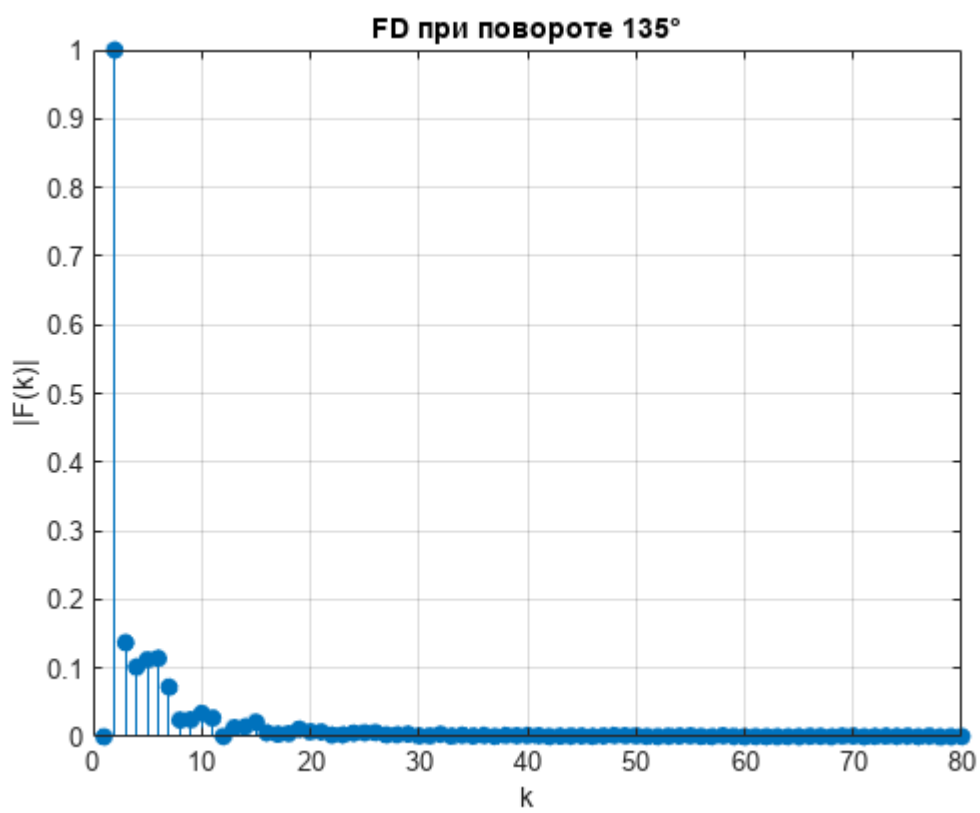
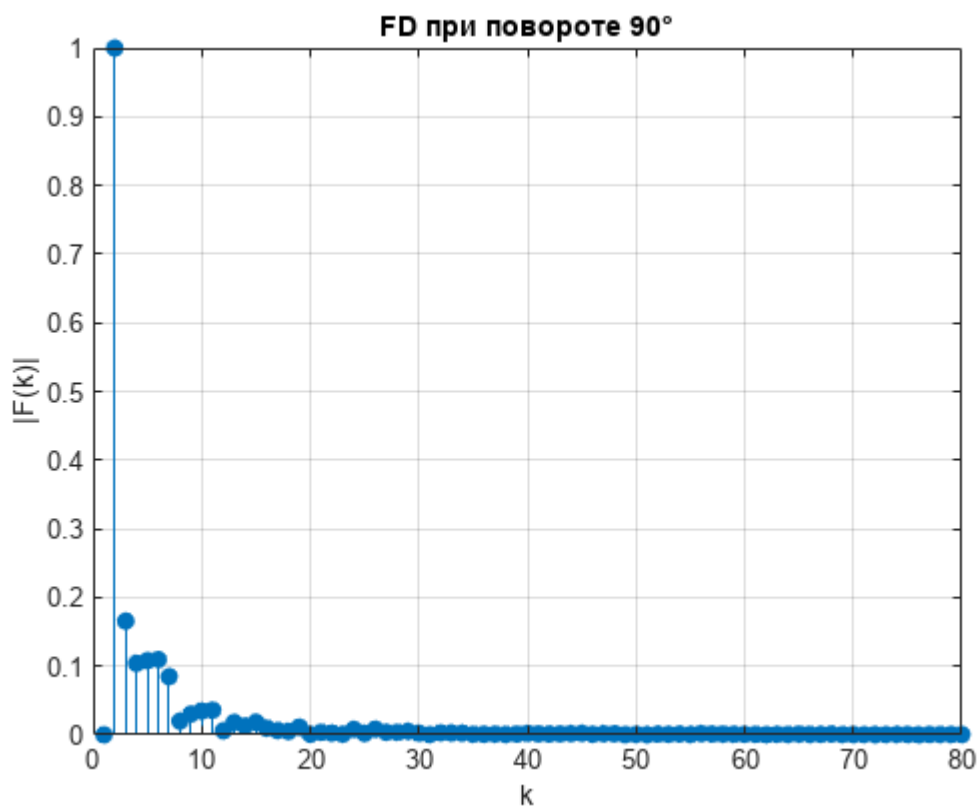
    FD_rot_all{i} = FD_rot;

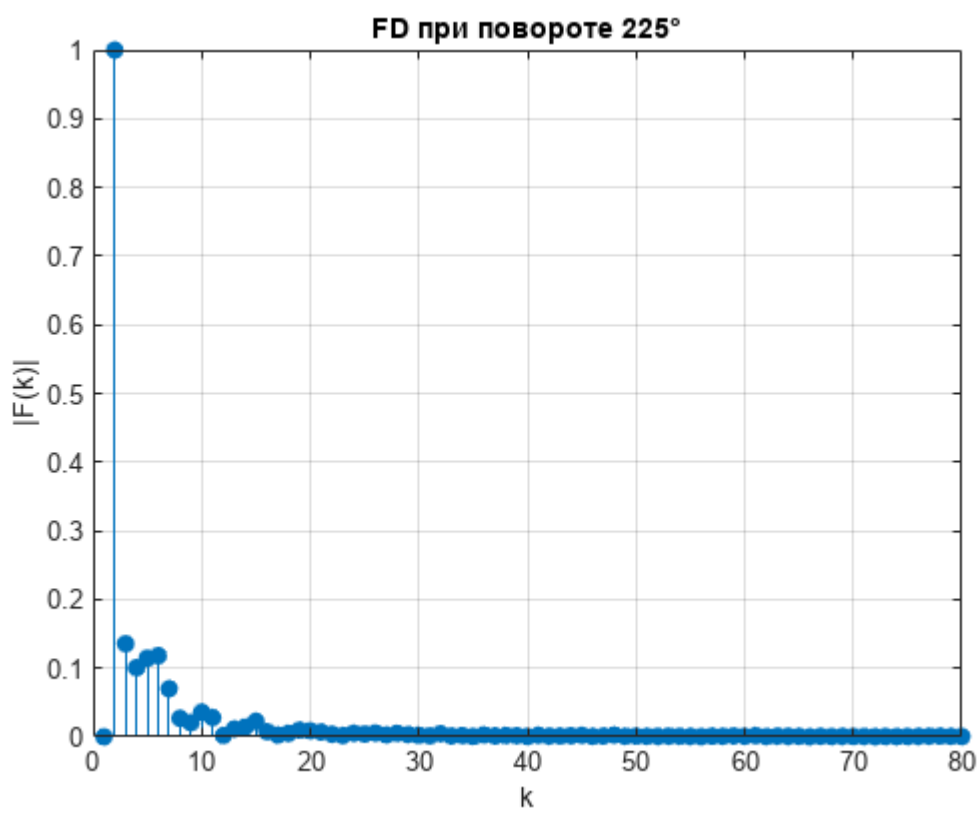
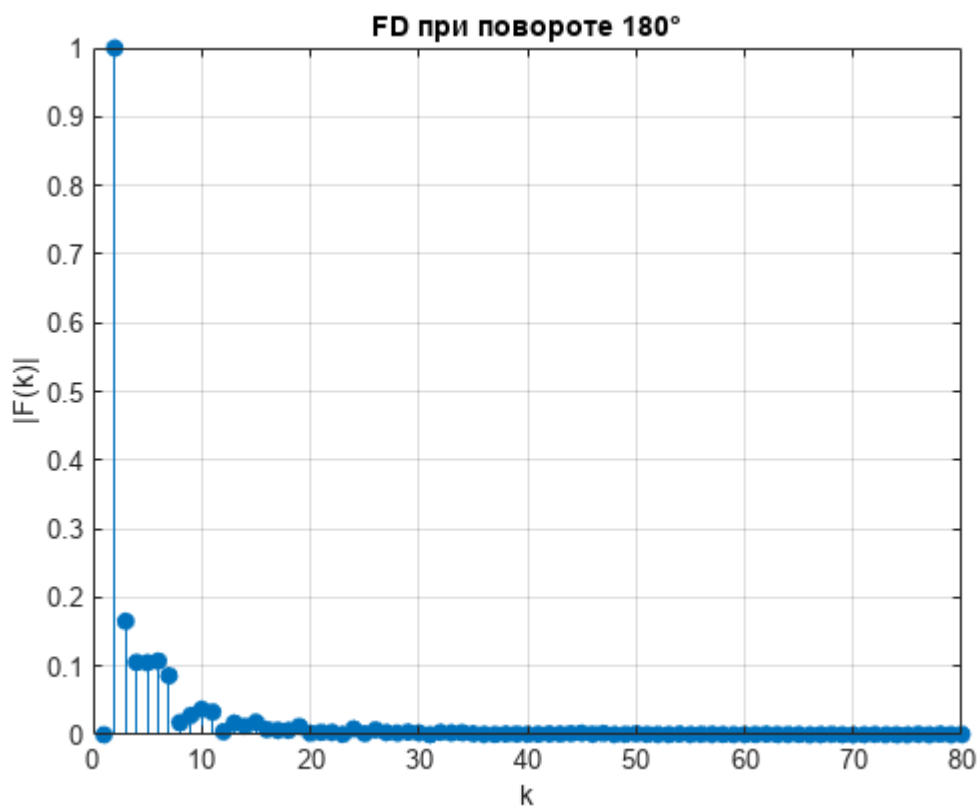
    % График для отчёта
    fig = figure("Color","w");
    stem(FD_rot(1:80), "filled"); grid on;
    title(sprintf("FD при повороте %d°", ang));
    xlabel("k"); ylabel("|F(k)|");

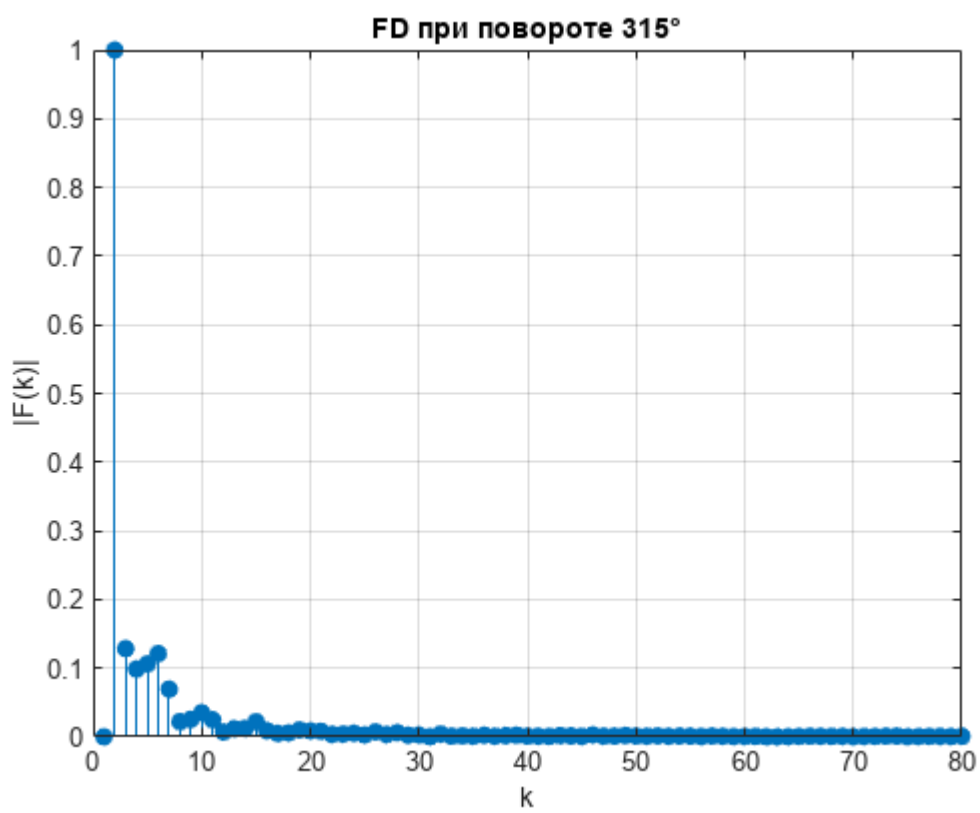
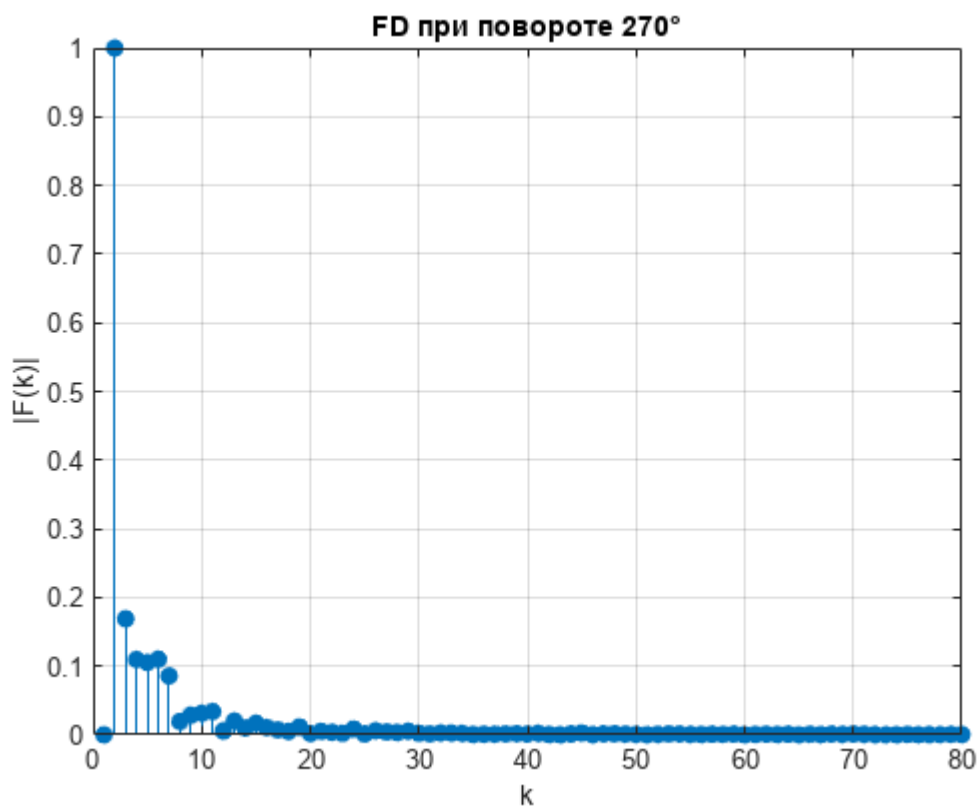
    exportgraphics(fig, fullfile(outDir, sprintf("FD_rot_%d.png", ang)), "Resolution", 200);
    close(fig);
end

```









Сравнение с исходным


```

fig = figure("Color","w");
hold on; grid on;

stem(FD(1:80), "k", "filled");

for i = 1:length(angles)
    if ~isempty(FD_rot_all{i})
        plot(FD_rot_all{i}(1:80), "--");
    end
end

title("Сравнение FD при разных поворотах");
legend(["original", string(angles)], "Location","northeastoutside");
xlabel("k"); ylabel("|F(k)|");

exportgraphics(fig, fullfile(outDir, "FD_rotation_compare.png"), "Resolution", 200);

```



```
close(fig);
```

Визуализация и сохранение

```

fig1 = figure("Color","w","Name","ЛР2 Вариант 5 – Сегментация и контур");
tiledlayout(fig1, 2, 3, "Padding","compact", "TileSpacing","compact");

nexttile; imshow(Ig, []); title("Полутонное исходное");
nexttile; imshow(IgAdj, []); title("Контраст (adapthisteq)");

```

```

nexttile; imshow(BW0, []); title(sprintf("Порог: T=%.3f (Otsu=%.3f)", T, T0));
nexttile; imshow(BWlink, []); title("После связывания маской");
nexttile; imshow(BWclean, []); title("Очистка области (fill/close)");
nexttile; imshow(BWedge2, []); title("Контур (Sobel + доработка)");

exportgraphics(fig1, fullfile(outDir, "step1_segmentation_contour.png"), "Resolution", 200);
close(fig1);

imwrite(Ig, fullfile(outDir, "00_gray.png"));
imwrite(IgAdj, fullfile(outDir, "01_contrast.png"));
imwrite(BW0, fullfile(outDir, "02_threshold.png"));
imwrite(BWlink, fullfile(outDir, "03_link_mask.png"));
imwrite(BWclean, fullfile(outDir, "04_region_clean.png"));
imwrite(BWedge2, fullfile(outDir, "05_edge_final.png"));

% Контур в координатах (как в методичке) и дескрипторы
fig2 = figure("Color","w","Name","ЛР2 Вариант 5 – Контур и Фурье-дескрипторы");
tiledlayout(fig2, 1, 2, "Padding","compact", "TileSpacing","compact");
nexttile;
plot(real(z), imag(z), "k"); axis equal; grid on;
set(gca, "YDir","reverse"); % чтобы было как координаты изображения
title("Контур как комплексный сигнал (x,y)");
xlabel("x"); ylabel("y");

nexttile;
stem(FD(1:min(Nshow, numel(FD))), "filled"); grid on;
title(sprintf("Фурье-дескрипторы (первые %d)", min(Nshow, numel(FD))));
xlabel("k"); ylabel("|F(k)| (норм.)");

exportgraphics(fig2, fullfile(outDir, "step2_fourier_descriptors.png"), "Resolution", 200);
close(fig2);

save(fullfile(outDir, "fourier_descriptors.mat"), "FD", "T0", "T", "edgeT");

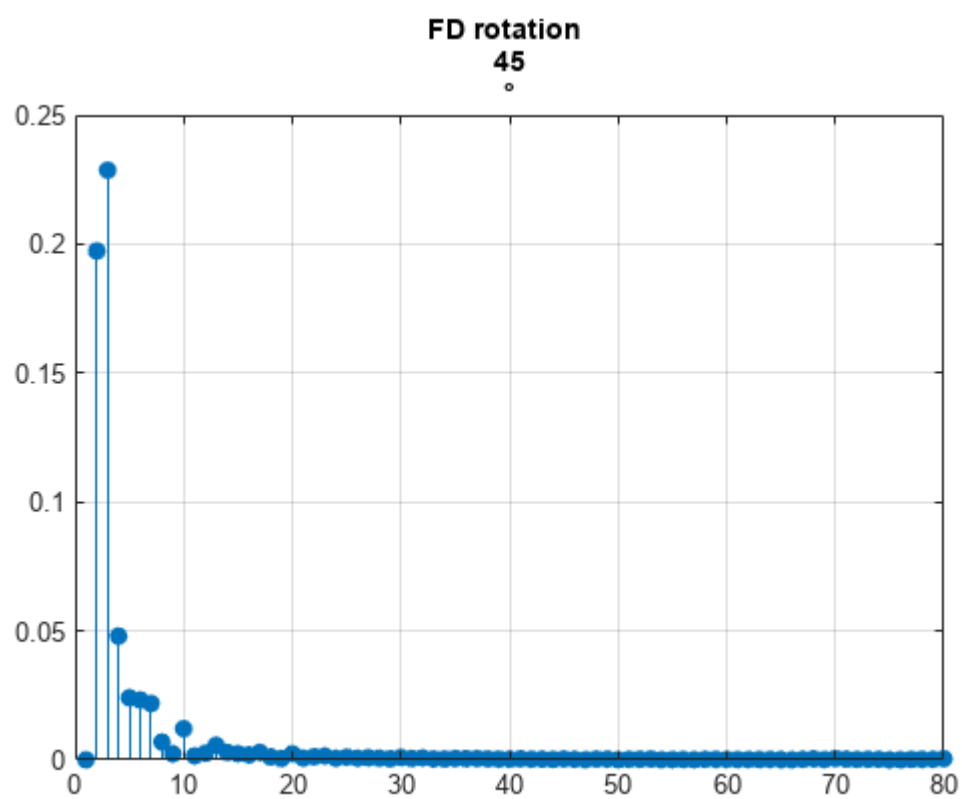
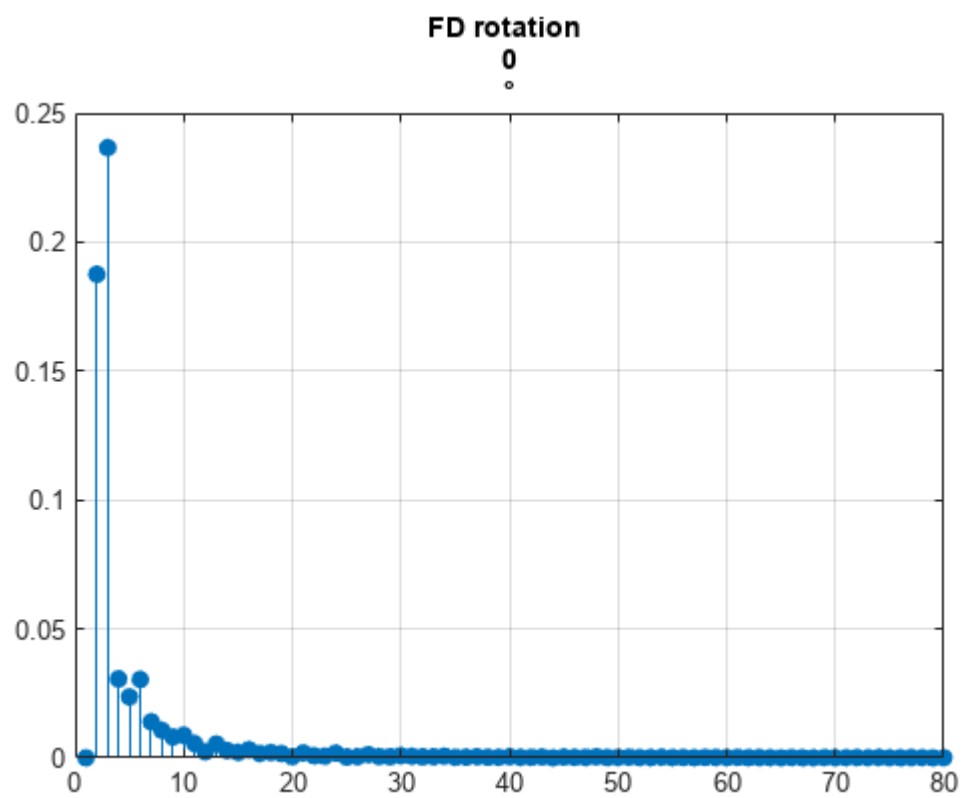
```

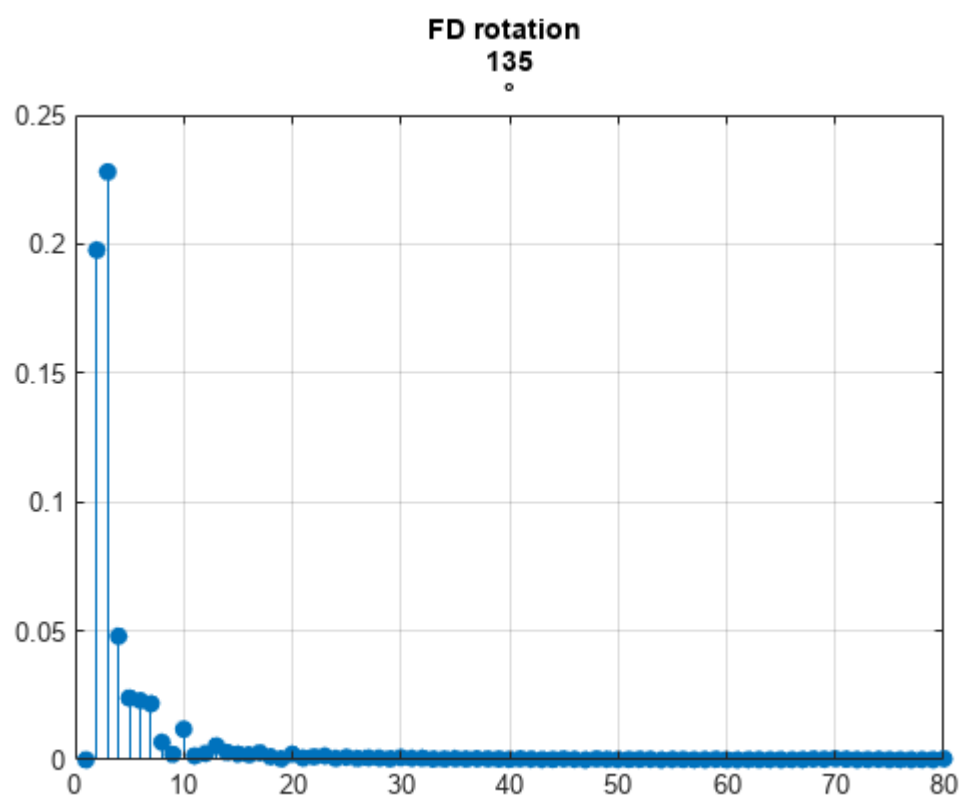
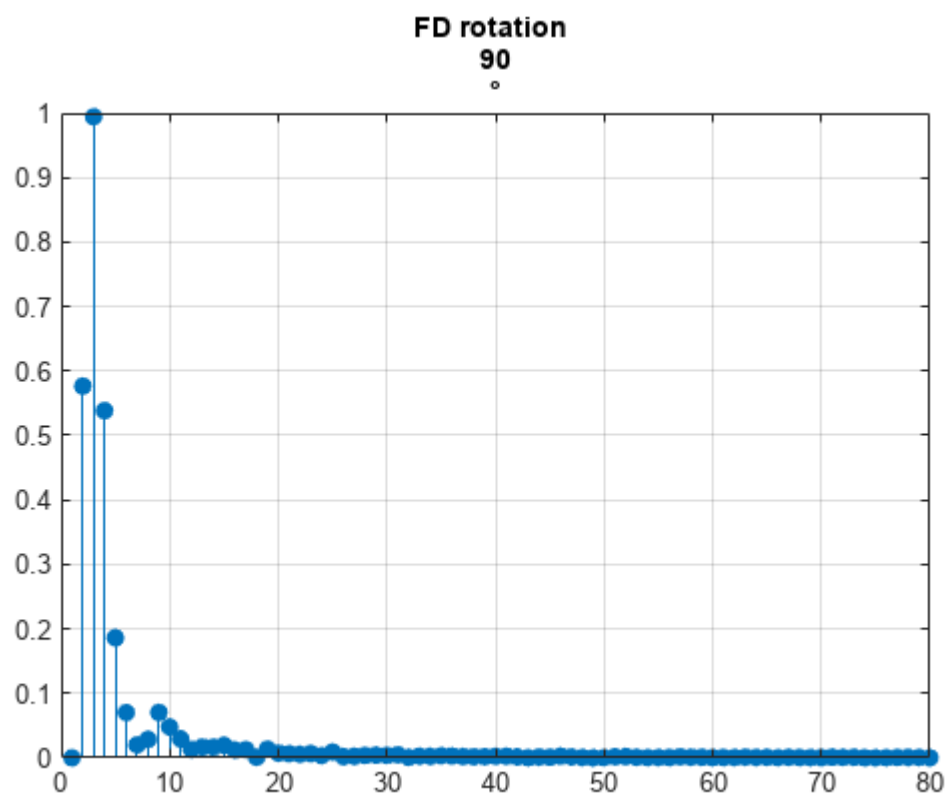
Дополнительно: если рядом есть символ из Paint, посчитать для него

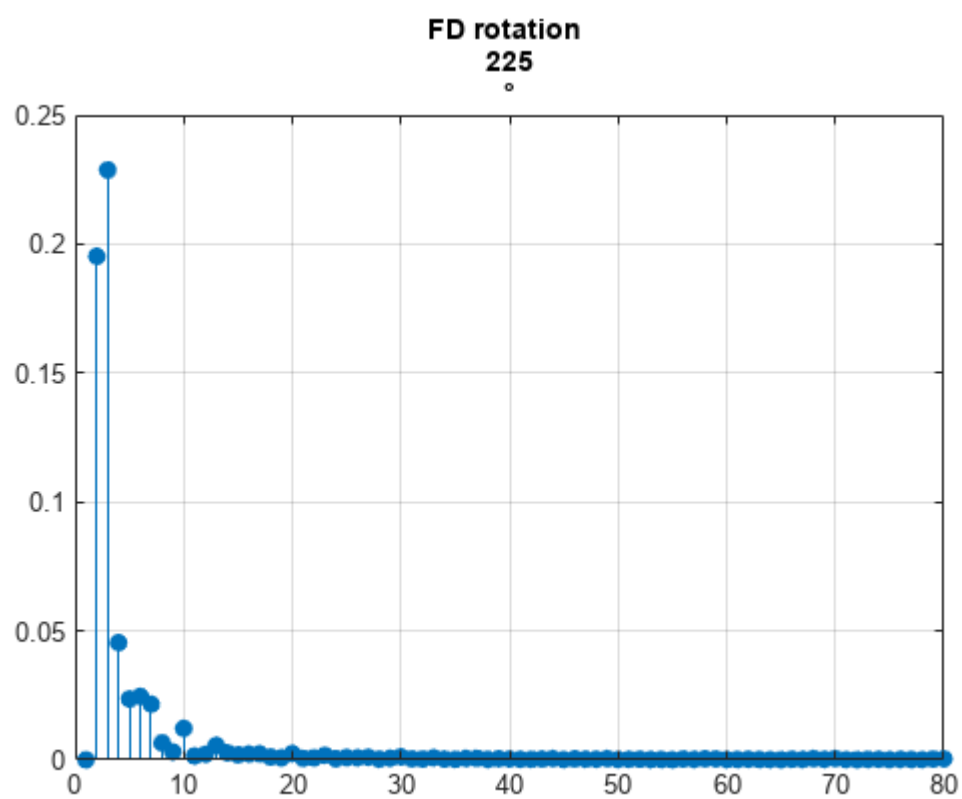
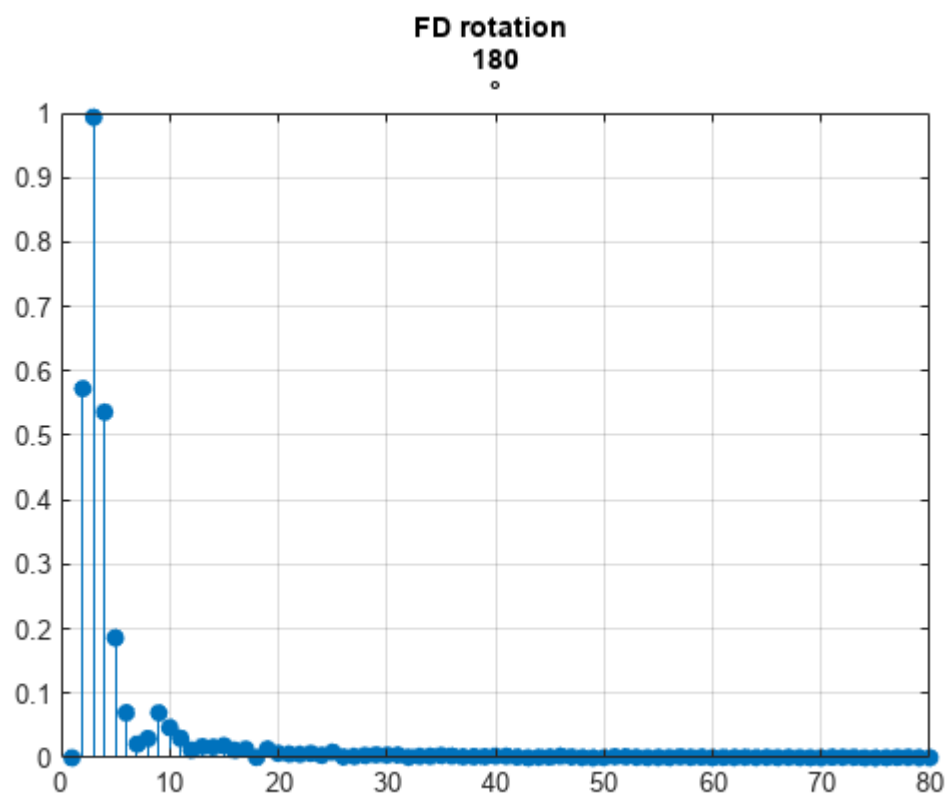
```

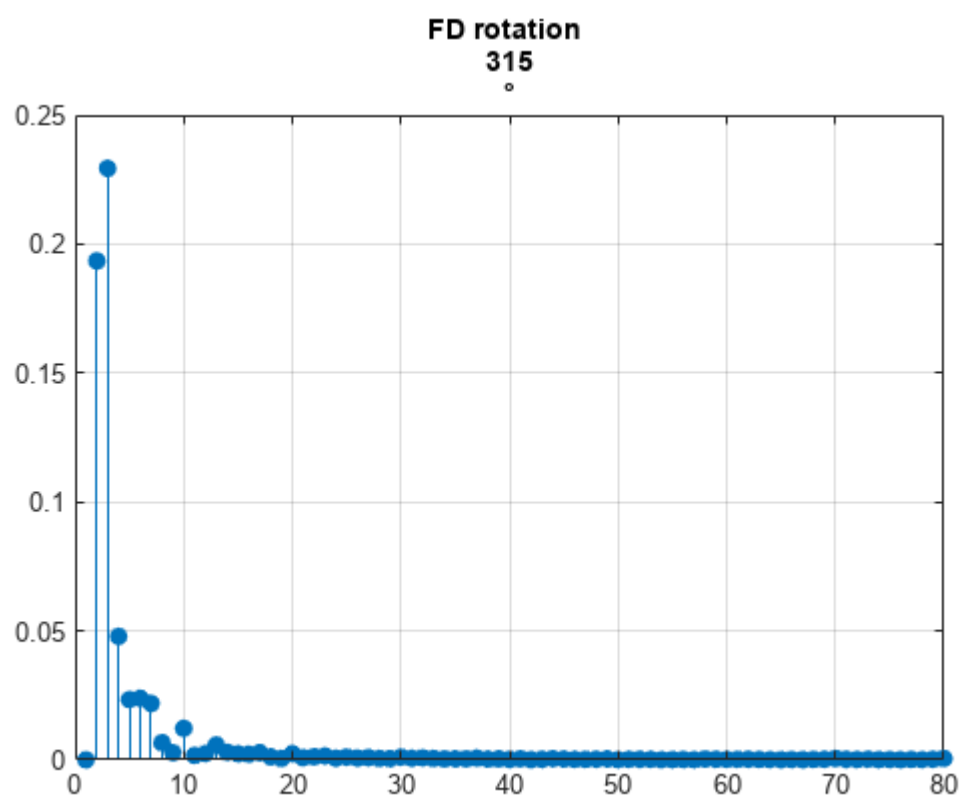
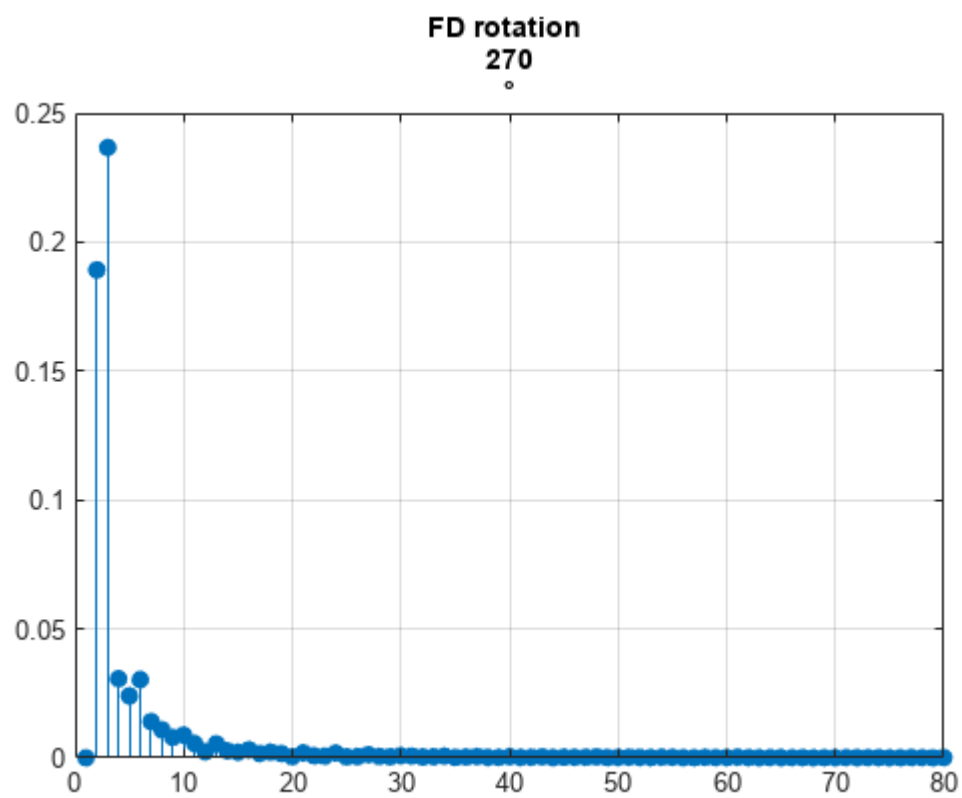
symPath = pickFirstExisting({fullfile(imgDir,"Symbol.png"), fullfile(imgDir,"Symbol.jpg"), fullfile(imgDir,"Symbol.tif")});
if ~isempty(symPath)
    runSymbolFourier(symPath, fullfile(outDir, "symbol"));
end

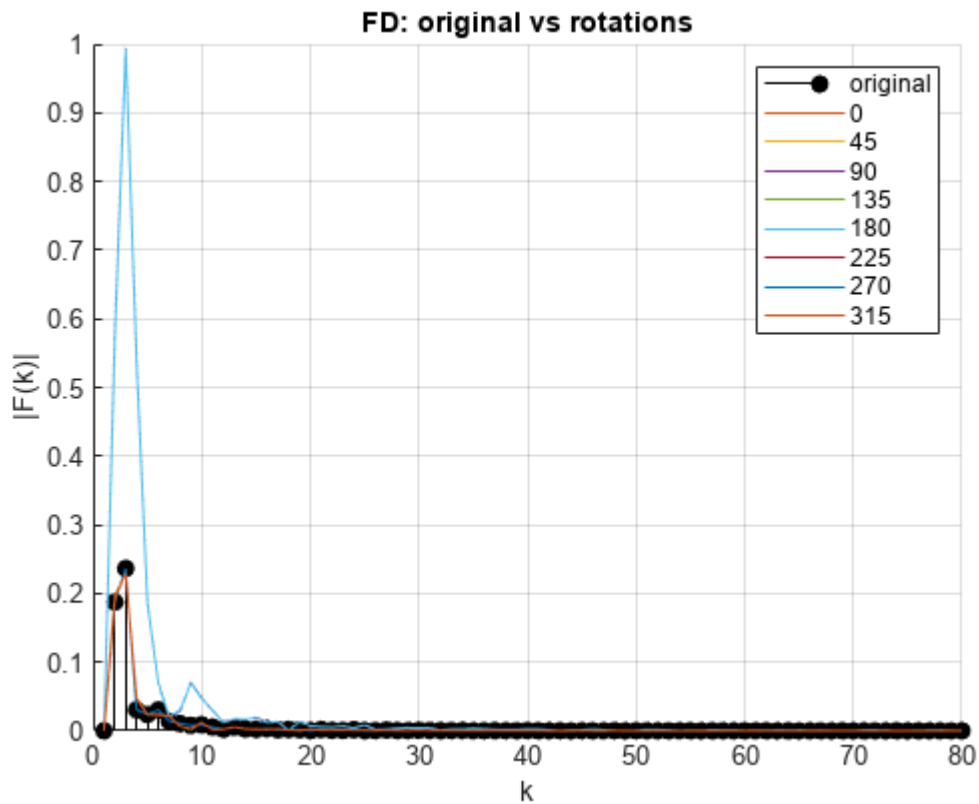
```











```
disp("Готово. Результаты в папке:");
```

Готово. Результаты в папке:

```
disp(outDir);
```

C:\Users\artem\AppData\Local\Temp\Editor_makux\Результаты

Локальные функции

```
function BW2 = linkByMask(BW, g, kMin)
% Связывание по маске (2g+1)x(2g+1): если сумма>=kMin -> 1
BW = logical(BW);
[m, n] = size(BW);
BW2 = BW;
for i = 1:m
    for j = 1:n
        if i>g && j>g && i<=m-g && j<=n-g
            T = BW(i-g:i+g, j-g:j+g);
            if sum(T(:)) >= kMin
                BW2(i,j) = 1;
            end
        end
    end
end
end
```

```

end

function BWbest = keepBestComponent(BW)
% Выбор компоненты: у самолёта центр близко к центру изображения,
% площадь большая, но обычно меньше, чем у "слитого" фона.
BW = logical(BW);
CC = bwconncomp(BW);
if CC.NumObjects == 0
    BWbest = BW;
    return;
end

stats = regionprops(CC, "Area", "Centroid", "Solidity");
sz = size(BW);
c0 = [sz(2)/2, sz(1)/2]; % [x,y] центр изображения

score = zeros(numel(stats), 1);
for i = 1:numel(stats)
    a = stats(i).Area; % площадь компоненты
    c = stats(i).Centroid; % центр компоненты
    d = hypot(c(1) - c0(1), c(2) - c0(2)); % расстояние до центра изображения
    s = stats(i).Solidity; % "плотность" компоненты

    % Простая и стабильная метрика: хотим крупную компоненту рядом с центром
    score(i) = (a * (0.4 + 0.6*s)) / (1 + 0.02*d);
end

[~, idx] = max(score);
BWbest = false(sz);
BWbest(CC.PixelIdxList{idx}) = true;
end

function [bound, z] = contourToComplex(BWedge)
% Берём самый длинный контур и переводим в комплексные координаты
CC = bwconncomp(BWedge);
if CC.NumObjects == 0
    error("Контур не найден: BWedge пустой. Попробуйте другой edgeT/порог.");
end
lens = cellfun(@numel, CC.PixelIdxList);
[~, idx] = max(lens);
mask = false(size(BWedge));
mask(CC.PixelIdxList{idx}) = true;

B = bwboundaries(mask, "noholes");
bound = B{1};
% bound(:,2) = x (col), bound(:,1) = y (row)
z = complex(bound(:,2), bound(:,1));
z = z - mean(z); % инвариантность к сдвигу
end

```



```

function p = pickFirstExisting(paths)
p = "";
for i = 1:numel(paths)
    if exist(paths{i}, "file") == 2
        p = paths{i};
        return;
    end
end
end

function runSymbolFourier(symPath, outBase)
I = im2double(imread(symPath));
if ndims(I) == 3, I = rgb2gray(I); end

% Часто в Paint символ чёрный на белом – приведём к "белый объект на чёрном"
if mean(I(:)) > 0.5
    I = 1 - I;
end

% Бинаризация + небольшая очистка
T0 = graythresh(I);
BW = I >= T0;
BW = bwareaopen(BW, 200);
BW = imfill(BW, "holes");
BW = imclose(BW, strel("disk", 2));

% Контур и дескрипторы – как в методичке:
% BW = edge(...); затем bwtraceboundary от стартовой точки.
BWedge = edge(BW, "sobel", 0.2);

% Стартовая точка контура (любой пиксель контура). Берём первый найденный.
[row, col] = find(BWedge, 1, "first");
if isempty(row)
    error("Символ: контур не найден. Попробуйте другой порог/очистку.");
end

% Обход контура (замкнутая линия) – получаем координаты boundary
BOUND = bwtraceboundary(BWedge, [row, col], "N");
if isempty(BOUND)
    error("Символ: bwtraceboundary не смог построить границу. Проверьте, что контур замкнут.");
end

% Комплексный сигнал контура (как в методичке: комплекс(row,col))
z = complex(BOUND(:, 1), BOUND(:, 2));
z = z - mean(z); % инвариантность к сдвигу

FD = abs(fft(z.')); % амплитуды Фурье (дескрипторы)
FD = FD / max(FD(:) + eps); % нормировка (инвариантность к масштабу)

```

===== Повороты символа =====

```
angles = 0:45:315;
FD_rot_all = cell(length(angles),1);

for i = 1:length(angles)
    ang = angles(i);

    BWrot = imrotate(BW, ang, "bilinear", "crop");
    BWedge_rot = edge(BWrot, "sobel", 0.2);

    [r, c] = find(BWedge_rot, 1);
    if isempty(r), continue; end

    BOUND_rot = bwtraceboundary(BWedge_rot, [r, c], "N");
    if isempty(BOUND_rot), continue; end

    zrot = complex(BOUND_rot(:,1), BOUND_rot(:,2));
    zrot = zrot - mean(zrot);

    FD_rot = abs(fft(zrot.'));
    FD_rot = FD_rot / (max(FD_rot(:)) + eps);

    FD_rot_all{i} = FD_rot;

    % Сохранение каждого поворота
    fig = figure("Color","w");
    stem(FD_rot(1:80), "filled"); grid on;
    title(["FD rotation ", num2str(ang), "°"]);

    exportgraphics(fig, fullfile(outBase, sprintf("FD_rot_%d.png", ang)), "Resolution", 200);
    close(fig);
end
```

===== Общий сравнительный график =====

```
fig = figure("Color","w");
hold on; grid on;

stem(FD(1:80), "k", "filled");

for i = 1:length(FD_rot_all)
    if ~isempty(FD_rot_all{i})
        plot(FD_rot_all{i}(1:80));
    end
end

title("FD: original vs rotations");
legend(["original", string(angles)]);
```

```

xlabel("k"); ylabel("|F(k)|");

exportgraphics(fig, fullfile(outBase, "FD_compare.png"), "Resolution", 200);
close(fig);

% Сохранить
if ~exist(fileparts(outBase), "dir"); mkdir(fileparts(outBase)); end
imwrite(BW, outBase + "_01_region.png");
imwrite(BWedge, outBase + "_02_edge.png");

% Графики как в методичке: контур и stem первых коэффициентов
figS = figure("Color", "w", "Name", "Symbol – Fourier descriptors");
tiledlayout(figS, 1, 2, "Padding", "compact", "TileSpacing", "compact");
nexttile;
plot(BOUND(:, 1), BOUND(:, 2), "k"); grid on; axis equal;
set(gca, "YDir", "reverse");
title("BOUND (bwtraceboundary)");
xlabel("row"); ylabel("col");

nexttile;
stem(FD(1:min(100, numel(FD))), "filled"); grid on;
title("FD (первые 100)");
xlabel("k"); ylabel("|F(k)|");

exportgraphics(figS, outBase + "_03_FD_plot.png", "Resolution", 200);
close(figS);

save(outBase + "_FD.mat", "FD", "T0", "BOUND", "row", "col");
end

```